QMANAGER

Backend Architecture Specification

The Gatekeeper & State-Cache Pattern

Target Hardware: Quectel RM551E-GL 5G Modem
Platform: OpenWRT (Embedded Linux)

Version 1.0 — February 2026

Table of Contents

1. System Constraints & Hardware Context

2. Core Philosophy

3. Component Architecture

3A. The Gatekeeper (qcmd)

3B. The Poller Daemon (qmanager_poller.sh)

3C. The State Cache (/tmp/qmanager_status.json)

3D. The Consumer Layer (CGI / Static Serving)

4. Command Classification: Short vs. Long

5. Two-Tier Polling Strategy

6. JSON Data Contract

7. Error Handling & Degraded Mode

8. Data Flow Diagram

9. Frontend Component Mapping

10. Implementation Build Order

11. Reference: AT Command Inventory

1. System Constraints & Hardware Context

QManager runs on a resource-constrained embedded Linux environment with specific hardware limitations that directly dictate the backend architecture.

Hardware

Parameter	Detail
Device	Quectel RM551E-GL 5G Modem
OS	Custom OpenWRT Build (Embedded Linux, Ash shell)
Storage	Flash (minimize writes — use /tmp/ RAM disk for volatile data)
Serial Interface	USB serial via /dev/ttyUSB2 (single-channel)
Bridge Tool	sms_tool (sole interface to modem AT commands)

> ⚠ **THE CRITICAL CONSTRAINT: The Single Pipe**
> The serial port to the modem is single-channel. It cannot handle concurrent AT commands. If two processes send commands simultaneously, the modem returns BUSY, garbage data, or sms_tool hangs entirely. Every AT command must pass through a serialized queue.

The Three Competing Actors

Three distinct subsystems need modem access simultaneously:

Actor	Purpose	Access Pattern
Dashboard Poller	Continuous signal/status updates for the UI	Every 2–30 seconds, multiple AT commands
User Terminal	Manual AT commands from the web terminal	Random, on-demand, expects fast response
Watchdog	Connectivity health checks	Periodic, infrequent

2. Core Philosophy

The architecture rests on two patterns that solve the concurrency problem:

The Gatekeeper (Mutex Lock): A wrapper script (qcmd) that forces all modem access through a file lock (flock). Only one actor can talk to the modem at any time. Everyone else waits in line.

The State Cache: A JSON file in RAM (/tmp/) that decouples reading from writing. The poller writes to the cache; the frontend reads from the cache. The UI never touches the modem directly, so it never freezes, even if the modem is busy or resetting.

Together, these patterns ensure stability (no race conditions), responsiveness (UI loads instantly from cache), and flash protection (all writes go to RAM).

3. Component Architecture

3A. The Gatekeeper — /usr/bin/qcmd

qcmd is the single entry point for all modem communication. Every AT command — from the poller, terminal, watchdog, or any future actor — must go through it.

Mechanism

- Uses flock (file locking) on /var/lock/qmanager.lock to serialize access.
- Classifies commands as short (<3s) or long (AT+QSCAN, ~2–3 minutes) and applies different timeout ceilings.
- Wraps sms_tool with the timeout utility to prevent indefinite hangs on modem firmware bugs.
- Records the lock holder's PID to detect and recover from stale locks (crashed processes).
- For long commands, sets a global flag (/tmp/qmanager_long_running) so other actors can voluntarily yield.

Stale Lock Recovery

A critical edge case: if sms_tool or the holding process crashes mid-command, the flock is never released. All actors queue indefinitely behind a dead lock. qcmd solves this by writing the holder's PID to a sidecar file. On timeout, the next caller checks if the PID is still alive. If not, it clears the stale lock and re-acquires.

Reference Implementation

```
#!/bin/sh
LOCK_FILE="/var/lock/qmanager.lock"
LOCK_FD=9
LONG_FLAG="/tmp/qmanager_long_running"
PID_FILE="/var/lock/qmanager.pid"
DEVICE="/dev/ttyUSB2"

is_long_command() {
    case "$1" in
        *QSCAN*|*QSCANFREQ*|*QFOTADL*) return 0 ;;
        *) return 1 ;;
    esac
}

exec 9>"$LOCK_FILE"

if is_long_command "$1"; then
    echo "$1" > "$LONG_FLAG"
    if ! flock -w 10 $LOCK_FD; then
        rm -f "$LONG_FLAG"
        echo '{"error":"modem_busy"}'
        exit 1
    fi
    echo $$ > "$PID_FILE"
    result=$(timeout 240 sms_tool -d "$DEVICE" at "$1" 2>/dev/null)
    flock -u $LOCK_FD
    rm -f "$LONG_FLAG" "$PID_FILE"

elif [ -f "$LONG_FLAG" ]; then
    echo '{"error":"modem_busy","detail":"scan_in_progress"}'
```

```
    exit 1

else
    if ! flock -w 5 $LOCK_FD; then
        # Check for stale lock
        if [ -f "$PID_FILE" ]; then
            OLD_PID=$(cat "$PID_FILE")
            if ! kill -0 "$OLD_PID" 2>/dev/null; then
                rm -f "$PID_FILE"
                flock -w 2 $LOCK_FD || { echo '{"error":"modem_busy"}'; exit 1; }
            fi
        fi
    fi
    echo $$ > "$PID_FILE"
    result=$(timeout 3 sms_tool -d "$DEVICE" at "$1" 2>/dev/null)
    flock -u $LOCK_FD
    rm -f "$PID_FILE"
fi

echo "$result"
```

3B. The Poller Daemon — qmanager_poller.sh

The poller is a background daemon that continuously gathers data from the modem and system, translates it into clean JSON, and writes it to the state cache.

"Sip, Don't Gulp" Strategy

A naive approach locks the modem, runs 10+ commands in a burst, then releases. This blocks the user's terminal for seconds. Instead, the poller sips: acquire lock, run one command, release, sleep briefly, repeat. This leaves gaps for the terminal and watchdog to slip in.

Long Command Awareness

Before each poll cycle, the poller checks for the /tmp/qmanager_long_running flag. If present (meaning AT+QSCAN or similar is running), the poller skips all modem commands but continues updating non-modem metrics (CPU, memory, traffic from /proc). This keeps the dashboard partially alive during long scans instead of going fully stale.

```
poll_cycle() {
    if [ -f /tmp/qmanager_long_running ]; then
        update_proc_metrics   # /proc/net/dev, loadavg, uptime
        write_cache           # Keep timestamp fresh
        return
    fi

    # Tier 1: Hot data (every cycle)
    poll_serving_cell
    sleep 0.1

    # Tier 2: Warm data (every N cycles)
    if [ $((cycle_count % 15)) -eq 0 ]; then
        poll_temperature
        sleep 0.1
        poll_carrier_info
    fi

    update_proc_metrics
    write_cache
}
```

3C. The State Cache — /tmp/qmanager_status.json

A single JSON file in RAM that serves as the single source of truth for the frontend.

Design Decisions

1. **RAM-resident:** /tmp/ on OpenWRT is a tmpfs (RAM disk). Writes are infinite and fast, with no flash wear.
2. **Atomic updates:** The poller writes to status.json.tmp first, then uses mv (rename, which is atomic on Linux) to replace status.json. The frontend never reads a half-written file.
3. **Staleness metadata:** Includes a timestamp and system_state field so the UI can detect stale data and show appropriate indicators.

3D. The Consumer Layer — CGI / Static File Serving

The frontend accesses modem data through two pathways:

GET /api/status.json (Dashboard): Zero latency. Does not touch the modem. The recommended approach is to serve the cache file directly as a static asset via uhttpd (symlink from /www/api/status.json to /tmp/qmanager_status.json). This avoids CGI fork overhead entirely. If authentication or CORS headers are required, a lightweight CGI script or uhttpd's Lua handler can wrap the file read.

POST /cgi-bin/api/terminal (User Commands): Passes the user's AT command to qcmd. Because the poller sips (sleeps between commands), the user's command can slip into the queue almost immediately. Returns the raw modem response as JSON.

4. Command Classification: Short vs. Long

Not all AT commands are equal. The gatekeeper must distinguish between sub-second commands and multi-minute operations to prevent system-wide denial of service.

Class	Lock Duration	Timeout	Examples	Behavior
Short	~200ms	3 seconds	AT+QENG, AT+QTEMP, AT+COPS?, AT+QCAINFO	Normal flock acquire → execute → release
Long	2–3 minutes	4 minutes	AT+QSCAN, AT+QSCANFREQ, AT+QFOTADL	Set global flag → poller yields → execute → clear flag

> ⚠ **Why This Matters**
> Without command classification, AT+QSCAN holds the flock for 2–3 minutes. During that window: the poller is blocked (dashboard freezes on stale data), the terminal is blocked (user commands time out), and the watchdog is blocked (may falsely trigger a modem restart during the scan). The long-command flag prevents this cascade.

Long Command List (Configurable)

Maintained in /etc/qmanager/long_commands.list so new slow commands can be added without modifying qcmd:

```
# /etc/qmanager/long_commands.list
QSCAN
QSCANFREQ
QFOTADL
```

Terminal Safety

The raw terminal in the web UI should not blindly pass long commands to qcmd without user awareness. Two approaches:

Recommended: The Cell Scanner page (/cellular/cell-scanner) is the only UI path that can trigger AT+QSCAN, via a dedicated endpoint with a progress indicator. The raw terminal blocks QSCAN with a message: “Use the Cell Scanner page for this command.”

Alternative: The terminal allows it but shows a warning modal: “This command takes 2–3 minutes and will pause live metrics. Continue?”

5. Two-Tier Polling Strategy

Not all data needs to be refreshed at the same rate. Polling carrier name every 2 seconds is wasteful when it changes once per session. The poller uses a two-tier strategy to minimize lock contention:

Tier 1 — Hot Data (Every 2 Seconds)

Data the user actively monitors. Changes frequently.

Source	Data Produced	Lock Required?
AT+QENG="servingcell"	Band, EARFCN, PCI, RSRP, RSRQ, SINR, Connection state	Yes
/proc/net/dev	Live RX/TX bytes (traffic calculation)	No
/proc/loadavg	CPU load average	No
/proc/uptime	Device uptime	No

Tier 1 holds the modem lock for only one AT command (~200ms) per cycle. The /proc reads require no lock. This keeps the lock window minimal for terminal responsiveness.

Tier 2 — Warm Data (Every 15–30 Seconds)

Data that rarely changes mid-session.

Source	Data Produced	Lock Required?
AT+QTEMP	Modem temperature	Yes
AT+COPS?	Carrier / operator name	Yes
AT+QNWINFO	Network type (LTE/NR/NSA/SA)	Yes
AT+QCAINFO	Carrier aggregation status	Yes
AT+CNUM	Phone number	Yes

Boot-Only Data

Some data is effectively static and only needs to be read once at poller startup: firmware version (AT+QGMR), IMEI (AT+CGSN), IMSI (AT+CIMI), ICCID (AT+QCCID). These are read once, cached, and never re-polled.

6. JSON Data Contract

This is the definitive schema for /tmp/qmanager_status.json. Every field maps directly to a React component in the frontend. This contract must be agreed upon before any backend or frontend implementation begins.

```
{
  "timestamp": 1707900000,
  "system_state": "normal",
  "modem_reachable": true,
  "last_successful_poll": 1707900000,
  "errors": [],

  "network": {
    "type": "5G",
    "sim_slot": 1,
    "carrier": "TMobile",
    "service_status": "optimal"
  },

  "lte": {
    "state": "connected",
    "band": "B3",
    "earfcn": 1850,
    "pci": 100,
    "rsrp": -95,
    "rsrq": -10,
    "sinr": 12.5
  },

  "nr": {
    "state": "connected",
    "band": "N41",
    "arfcn": 499200,
    "pci": 200,
    "rsrp": -88,
    "rsrq": -9,
    "sinr": 15.0
  },

  "device": {
    "temperature": 45,
    "cpu_usage": 23,
    "memory_used_mb": 3200,
    "memory_total_mb": 8192,
    "uptime_seconds": 45910,
    "conn_uptime_seconds": 19412,
    "firmware": "RM551EGL00AAR01A04M8G",
    "imei": "866792052000123",
    "imsi": "310260123456789",
    "iccid": "8901260420001234567",
    "phone_number": "+15551234567",
    "lte_category": "Cat 20",
    "mimo": "LTE 4x4 | NR 4x4"
  },

  "traffic": {
    "rx_bytes_per_sec": 1562500,
    "tx_bytes_per_sec": 375000
```

```
  }
}
```

Schema Rules

1. Signal values (rsrp, rsrq, sinr) are always numbers, never strings with units. The frontend handles formatting and unit display.
2. Band names use standard 3GPP notation: "B3" for LTE Band 3, "N41" for NR Band 41.
3. The timestamp field is Unix epoch (seconds). Updated on every write cycle.
4. The errors array contains string codes, not human-readable messages. The frontend maps codes to user-facing text.
5. Traffic values are raw bytes per second. The frontend converts to Mbps/Kbps as needed.

7. Error Handling & Degraded Mode

The poller must never crash. When the modem becomes unresponsive or returns errors, the cache must reflect the degraded state with valid JSON so the frontend can display appropriate indicators.

Error Scenarios

Scenario	Detection	Cache Response
SIM ejected	AT+CPIN? returns ERROR	modem_reachable: true, errors: ["sim_not_inserted"], lte.state: "error"
Modem unresponsive	sms_tool timeout (exit code != 0)	modem_reachable: false, last_successful_poll preserved
Airplane mode	AT+QENG returns NOCONN	service_status: "no_service", signal fields set to null
sms_tool crash	Lock holder PID is dead	qcmd recovers stale lock; poller retries next cycle
Long scan active	/tmp/qmanager_long_running exists	system_state: "scan_in_progress", non-modem metrics continue

Key Principle: Never Lie to the User
Stale data with a green "connected" indicator is worse than showing "Last updated: 1 min ago" with an amber badge. The cache must always include enough metadata for the frontend to distinguish fresh data from stale data.

Degraded Cache Example

```
{
  "timestamp": 1707900120,
  "system_state": "degraded",
  "modem_reachable": false,
  "last_successful_poll": 1707899940,
  "errors": ["modem_timeout"],

  "network": {
    "type": "5G",
    "service_status": "unknown"
  },
  "lte": { "state": "unknown" },
  "nr": { "state": "unknown" },

  "device": {
    "temperature": null,
    "cpu_usage": 18,
    "memory_used_mb": 3100,
    "uptime_seconds": 46020
  },
  "traffic": {
    "rx_bytes_per_sec": 0,
    "tx_bytes_per_sec": 0
  }
}
```

8. Data Flow Diagram

```
   [USER BROWSER]                    [OPENWRT SYSTEM]
         |                                  |
(1) Load Dashboard  ==================>  [uhttpd]
         |                                  |
         |                          Reads /tmp/status.json (RAM)
         |                          Instant. No modem contact.
         |
         |
(2) Send AT Command  ================>  [CGI Script]
         |                                  |
         |                            Calls /usr/bin/qcmd
         |                                  |
         |                                  v
 [Background Poller]  ============>  [GATEKEEPER (flock)]
 (writes status.json)                       |
         |                                  |
 [Watchdog]  =====================>         |
                                            | (one at a time)
                                            v
                                       [sms_tool]
                                            |
                                            v
                                      [RM551E-GL]



 LONG COMMAND FLOW (AT+QSCAN):

 [Cell Scanner Page]  ===>  qcmd sets /tmp/qmanager_long_running
                                 |
                            Poller sees flag -> skips modem,
                            continues /proc metrics only
                                 |
                            qcmd acquires lock -> sms_tool AT+QSCAN
                            (2-3 min)
                                 |
                            qcmd clears flag -> poller resumes
```

9. Frontend Component Mapping

Each React component on the Home dashboard maps to specific fields in the JSON cache. This table is the definitive reference for both frontend and backend developers.

Component	File	JSON Path(s)	Polling Tier
Network Status	network-status.tsx	network.type, network.sim_slot, network.carrier, network.service_status	Tier 1
4G Primary Status	lte-status.tsx	lte.state, lte.band, lte.earfcn, lte.pci, lte.rsrp, lte.rsrq, lte.sinr	Tier 1
5G Primary Status	nr-status.tsx	nr.state, nr.band, nr.arfcn, nr.pci, nr.rsrp, nr.rsrq, nr.sinr	Tier 1
Device Status	device-status.tsx	device.firmware, device.imei, device.imsi, device.iccid, device.phone_number, device.lte_category, device.mimo	Boot-only
Device Metrics	device-metrics.tsx	device.temperature, device.cpu_usage, device.memory_*, device.uptime_seconds, traffic.*	Tier 1 (/proc) + Tier 2 (temp)
Signal History	signal-history.tsx	lte.rsrp, lte.sinr, nr.rsrp, nr.sinr (accumulated client-side)	Tier 1

10. Implementation Build Order

The following sequence minimizes wasted effort and ensures each layer is testable before the next depends on it.

1. **Define the JSON Contract.** Finalize the schema from Section 6. Create a sample status.json with realistic values. Both frontend and backend work against this contract.
2. **Build qcmd (Gatekeeper).** Implement lock acquisition, stale lock recovery, command classification, and timeout wrapping. Test by opening two SSH sessions and firing concurrent commands. Verify one waits, both return valid data, and a killed qcmd does not permanently lock the system.
3. **Build the Poller (Tier 1 Only).** Start with AT+QENG and /proc reads only. Parse the output into the JSON contract format. Write to /tmp/qmanager_status.json with atomic mv. Verify valid JSON on every cycle.
4. **Wire the API Endpoint.** Set up the static file symlink or CGI wrapper. Verify the browser can fetch /api/status.json and receives valid data.
5. **Connect the React Frontend.** Replace hardcoded values in LTEStatusComponent, NrStatusComponent, DeviceMetricsComponent, and NetworkStatusComponent with data from the fetch response. Implement a polling hook (e.g., useInterval at 2s).
6. **Add Tier 2 Polling.** Backfill temperature, carrier, CA status, and other slow-changing fields. Wire DeviceStatus component to boot-only data.
7. **Implement Error Handling.** Add degraded-mode JSON output. Test by ejecting SIM, disabling modem, and simulating sms_tool crashes. Verify the UI shows appropriate error indicators.
8. **Add Long Command Support.** Wire the Cell Scanner page to the dedicated scan endpoint. Verify the poller yields gracefully and the dashboard shows "scan in progress" state.

11. Reference: AT Command Inventory

Complete list of AT commands used by QManager, their purpose, expected response time, and polling tier assignment.

AT Command	Purpose	Response Time	Tier	Notes
AT+QENG="servingcell"	Primary/serving cell info (band, EARFCN, PCI, RSRP, RSRQ, SINR)	~200ms	Tier 1	Most critical command. Complex CSV output requires careful parsing.
AT+QTEMP	Modem internal temperature	~100ms	Tier 2	Returns multiple sensor readings.
AT+COPS?	Registered carrier name and access tech	~200ms	Tier 2	Changes only on cell reselection.
AT+QNWINFO	Current network type (LTE/NR-NSA/NR-SA)	~100ms	Tier 2	Useful for determining NSA vs SA.
AT+QCAINFO	Carrier aggregation component carriers	~200ms	Tier 2	Returns multiple lines for each CC.
AT+CNUM	SIM phone number (MSISDN)	~100ms	Boot	Static per SIM session.
AT+QGMR	Firmware version string	~100ms	Boot	Never changes at runtime.
AT+CGSN	Device IMEI	~100ms	Boot	Hardware identifier. Static.
AT+CIMI	SIM IMSI	~100ms	Boot	Identifies SIM. Static per session.
AT+QCCID	SIM ICCID	~100ms	Boot	SIM card serial. Static.
AT+CPIN?	SIM status (READY/ERROR)	~100ms	Tier 2	Used for SIM ejection detection.
AT+QSCAN	Full cell scan (all visible cells)	2–3 min	On-demand	LONG COMMAND. Requires flag-based coordination.

End of Document

QManager Backend Architecture Specification v1.0